Unit – IV

Mining Association Rules in Large Databases

Mining Association Rules in Large Databases

- **Association rule mining finds interesting association or correlation relationships among a large set of data items**

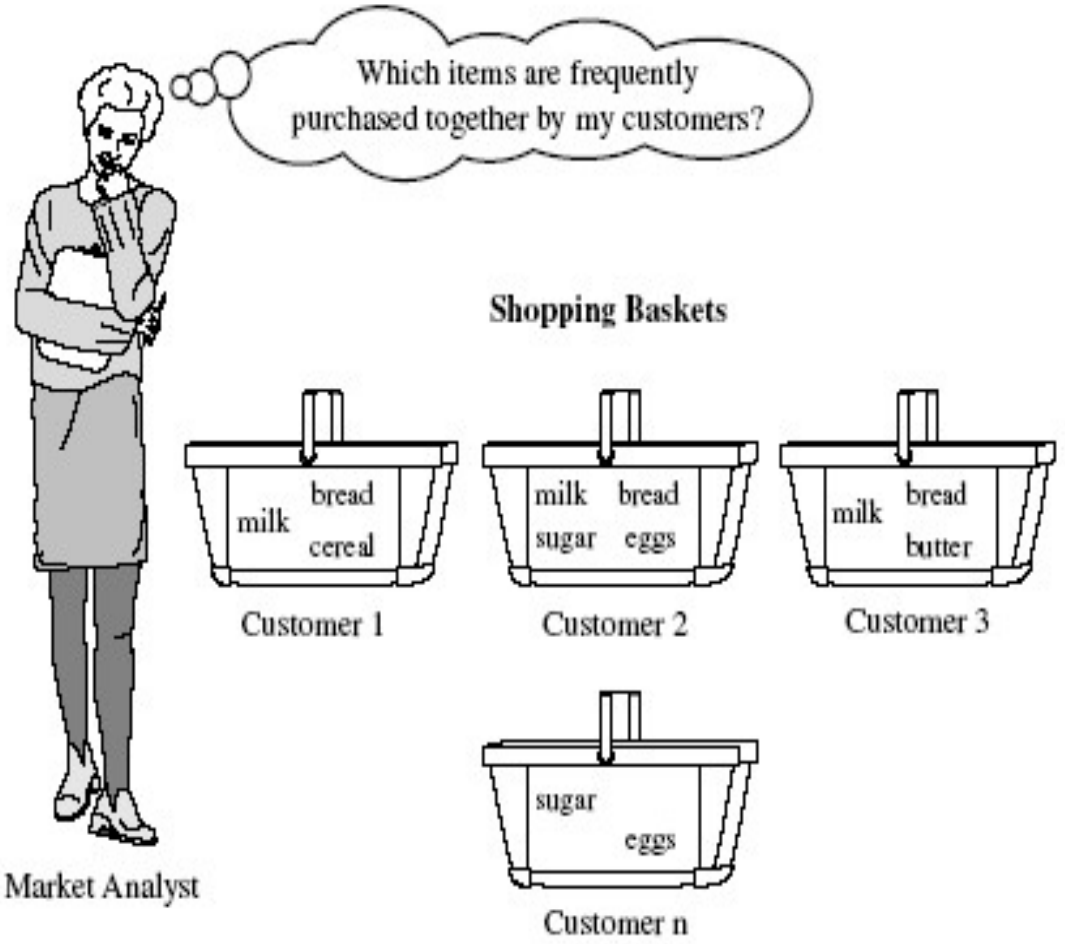
- The discovery of interesting association relationships among huge amount of business transaction records can help in many business decision making process, such as catalog design, cross marketing, and loss-leader analysis.

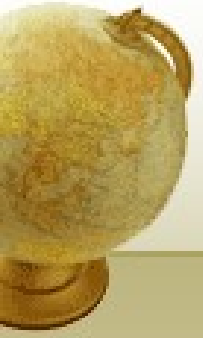
- A typical example of association rule mining is **market basket analysis**.

- This process analyzes customers buying habits by finding associations between different items that customers place in their shopping habits.

Mining Association Rules in Large Databases

- The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by the customers.
- For instance, if customers are buying milk, how likely are they to also buy bread (and which kind of bread) on the same trip to the supermarket.
- Such information can lead to increased sales by helping retailers do selective marketing and plan their shelf space.
- For ex, placing milk and bread within close proximity may further encourage the sale of these items together within single visits to the store.


Which items are frequently purchased together by my customers?
Shopping Baskets
milk
bread
cereal
Customer 1
milk
bread
sugar
eggs
Customer 2
milk
bread
butter
Customer 3
sugar
eggs
Customer n
Market Analyst



Market Basket Analysis

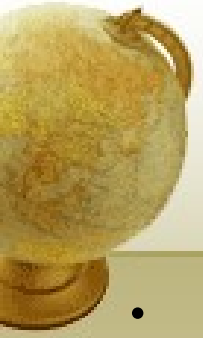
A motivating example for Association Rule Mining

- Suppose, as manager of AllElectronics branch you would like to learn more about the buying habits of your customers. Specially you wonder *"Which groups or sets of items are customers likely to purchase on a given trip to the store"*

- To answer this question, market basket analysis may be performed on the retail data of customer transactions at your store.

- In one strategy items that are frequently purchased together can be placed in close proximity in order to further encourage the sale of such items together.

- If customers who purchase computers also tend to buy financial management software at the same time, then placing the hardware display close to the software display may help to increase the sales of both of these items.



Market Basket Analysis

A motivating example for Association Rule Mining

- In an alternative strategy, placing hardware and software at opposite ends of the store may entice customers who purchase such items to pick up other items along the way.

- For instance, after deciding on an expensive computer, a customer may observe security systems, while heading towards management software.

- If we think of the universe as the set of items available at a store, then each item has a Boolean Variable representing the presence or absence of the item.

- Each basket can be represented by a Boolean Vector of values assigned to variables.
- The Boolean Vectors can be analyzed for buying patterns that reflect items that are frequently associated or purchased together

Market Basket Analysis

A motivating example for Association Rule Mining

- These patterns can be represented in the form of **association rules.**

- For ex, the information that customers who purchase computers tend to buy financial management software at the same time is represented in the Association Rule below

computer => financial_management_software

[support = 2%, confidence = 60%]

- **Basic Concepts**

  support (A=>B) = P (A Union B)

  confidence (A=>B) = P(B/A) = Support(A u B)/Support(A)

- How are association rules mined by from large databases?.
- Association rule mining is a two step process :

1. **Find all frequent itemsets** : By definition, each of these itemsets will occur atleast as frequently as a pre-determined minimum support count

2. **Generate strong association rules from the frequent itemsets** : By definition, these rules must satisfy minimum support and minimum confidence.

Association Rule Mining : A Road Map

- **Association rules can be classified in various ways, based on the following criteria**

- **Based on the types of values handled in the rule :** If a rule concerns associations between the presence or absence of items, it is a **Boolean Association Rule.**

For Ex :

computer => financial_management_software

[support = 2%, confidence = 60%]

Association Rule Mining : A Road Map

- If a rule describes associations between quantitative items or attributes, then it is a **quantitative association rule.**

- In these rule, quantitative values for items or attributes are partitioned into intervals. The following rule is an ex of a quantitative association rule, where X is a variable representing a customer

age (x, "30..39") ^ income (x, "42..48K")
→buys (x, high resolution TV)

Association Rule Mining : A Road Map

- **Based on the dimensions of data involved in the rule** : If the items or attributes in an association rule reference only one dimension, then it is a single dimensional association rule.

- Note that Rule

  computer => financial_management_software

  [support = 2%, confidence = 60%]

  can be rewritten as

  buys (x, "computer") → buys (x, "financial software")

Association Rule Mining : A Road Map

Note that Rule

computer => financial_management_software

[support = 2%, confidence = 60%]

is a single dimensional association rule since it refers to only one dimension, ***buys***. If a rule references two or more dimensions, such as the dimensions buys, time_of_transaction, and customer_category, then it is a multidimensional association rule.

age (x, "30..39") ^ income (x, "42..48K")

→buys (x, high resolution TV)

The above rule is considered a multidimensional association rule since it involves three dimensions, age, income, and buys

Association Rule Mining : A Road Map

- **Based on the levels of abstractions involved in the rule set :** Some methods for association rule mining can find rules at differing levels of abstraction. For ex, suppose that a set of association rules mined include the following rules.

  age(x, "30..39") →buys (x, "laptop computer") a
  age(x, "30..39") →buys (x, "computer") b

  In rules a and b the items bought are referenced at different levels of abstraction (eg. "computer" is a higher-level abstraction of "laptop computer"

  We refer to the rule set mined as consisting of **multilevel association rules.**

Types of Association Rule Mining

- Boolean vs. quantitative associations

  (Based on the types of values handled)
  - buys(x, “computer”) → buys(x, “financial software”) [2%, 60%]
  - age(x, “30..39”) ^ income(x, “42..48K”) →buys(x, “PC”) [1%, 75%]

- Single dimension vs. multiple dimensional associations
  - buys(x, “computer”) → buys(x, “financial software”) [2%, 60%]
  - age(x, “30..39”) ^ income(x, “42..48K”) →buys(x, “PC”) [1%, 75%]

Types of Association Rule Mining

- Single level vs. multiple-level analysis
  - What brands of beers are associated with what brands of diapers?
- Various extensions
  - Correlation, causality analysis
    - Association does not necessarily imply correlation or causality
  - Constraints enforced
    - E.g., small sales (sum < 100) trigger big buys (sum > 1,000)?

- Let us consider the following set of transactions in a bookshop.

  t1 := {ANN,CC,TC,CG}

  t2:= {CC,D,CG}

  t3 := {ANN,CC,TC,CG}

  t4:= {ANN,CC,D,CG}

  t5:= {ANN,CC,D,TC,CG}

  t6:= {CC,D,TC}
- I={ANN,CC,D,TC,CG} and T:={t1,t2,t3,t4,t5,t6}

Association Rule: Basic Concepts

- Given:

(1) database of transactions,

(2) each transaction is a list of items (purchased by a customer in a visit)

- Find: all rules that correlate the presence of one set of items with that of another set of items
  - E.g., 98% of people who purchase tires and auto accessories also get automotive services done

Association Rule Definitions

- ***Set of items:*** $I=\{I_1,I_2,\ldots,I_m\}$
- ***Transactions:*** D = {t1, t2,.., tn} be a set of transactions, where a transaction,t, is a set of items
- ***Itemset:*** $\{I_{i1},I_{i2}, \ldots, I_{ik}\} \subseteq I$
- ***Support of an itemset:*** Percentage of transactions which contain that itemset.
- ***Large (Frequent) itemset:*** Itemset whose number of occurrences is above a threshold.

Rule Measures: Support & Confidence

- An association rule is of the form : X → Y where X, Y are subsets of I, and X INTERSECT Y = EMPTY
- Each rule has two measures of value: support, and confidence.
- Support indicates the frequencies of the occurring patterns, and confidence denotes the strength of implication in the rule.
- The support of the rule X → Y is support (X UNION Y) c is the CONFIDENCE of rule X → Y if c% of transactions that contain X also contain Y, which can be written as:
  - **support(X UNION Y)/support(X)**

Association Discovery

- Given a user specified minimum support (called MINSUP) and minimum confidence (called MINCONF), an important
- PROBLEM is to find all high confidence, large itemsets (frequent sets, sets with high support). (where support and confidence are larger than minsup and minconf).
- This problem can be decomposed into two subproblems:
- 1. Find all large itemsets: with support > minsup (frequent sets).
- 2. For a large itemset, X and B $\in$X (or Y $\subset$ X) , find those rules, X\{B} => B (X-Y → Y) for which confidence > minconf.

Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases.

The Apriori Algorithm: Basics

- The Apriori Algorithm is an influential algorithm for mining frequent item sets for Boolean association rules. The name of algorithm is based on the fact that algorithm uses prior knowledge of frequent itemset properties.
- Key Concepts :
  - Frequent Itemsets: The sets of item which has minimum support (denoted by L_i for i^{th}-Itemset).
  - Apriori Property: Any subset of frequent itemset must be frequent.
  - Join Operation: To find L_k , a set of candidate k-itemsets is generated by joining L_{k-1} with itself.

The Apriori Algorithm in a Nutshell

- Find the *frequent itemsets*: the sets of items that have minimum support
  - A subset of a frequent itemset must also be a frequent itemset
    - i.e., if {*AB*} is a frequent itemset, both {*A*} and {*B*} should be a frequent itemset
  - Iteratively find frequent itemsets with cardinality from 1 to *k* *(k*-itemset*)*
- Use the frequent itemsets to generate association rules.

The Apriori Algorithm : Pseudo code

- Join Step: C_k is generated by joining $L_{k-1}$with itself
- Prune Step: Any (k-1)-itemset that is not frequent cannot be a subset of a frequent k-itemset
- Pseudo-code:

  C_k: Candidate itemset of size k

  L_k : frequent itemset of size k

```
L1 = {frequent items};
for (k = 1; Lk !=∅; k++) do begin
    Ck+1 = candidates generated from Lk;
   for each transaction t in database do
            increment the count of all candidates in Ck+1 that are contained
          in t
   Lk+1  = candidates in Ck+1 with min_support
   end
return ∪k Lk;
```

The Apriori Algorithm: Example

TID	List of Items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2 ,I3, I5
T900	I1, I2, I3

- Consider a database, D , consisting of 9 transactions.
- Suppose min. support count required is 2 (i.e. min_sup = 2/9 = 22 %)
- Let minimum confidence required is 70%.
- We have to first find out the frequent itemset using Apriori algorithm.
- Then, Association rules will be generated using min. support & min. confidence.

Step 1: Generating 1-itemset Frequent Pattern

Scan D for count of each candidate

Itemset	Sup.Count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

C_1

Compare candidate support count with minimum support count

Itemset	Sup.Count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

L_1

- In the first iteration of the algorithm, each item is a member of the set of candidate.
- The set of frequent 1-itemsets, L_1 , consists of the candidate 1-itemsets satisfying minimum support.

Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L2 , the algorithm uses L1 *Join* L1 to generate a candidate set of 2-itemsets, C2.
- Next, the transactions in D are scanned and the support count for each candidate itemset in C2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L2 , is then determined, consisting of those candidate 2-itemsets in C2 having minimum support.
- Note: We haven't used Apriori Property yet.

L1 = {I1,I2,I3,I4,I5}

Since L2 = L1 join L1 then

{I1,I2,I3,I4,I5} join {I1,I2,I3,I4,I5}

It becomes

[{I1,I2} {I1,I3}, {I1,I4}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}, {I3,I4} {I3,I5}, {I4,I5}].

Now we need to check the frequent item sets with min support count

Then we get

L2= [{I1,I2} {I1,I3}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}].

Similarly We do it for L3

Step 2: Generating 2-itemset Frequent Pattern

Generate C_2 candidates from L_1

Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}

C_2

Scan D for count of each candidate

Itemset	Sup. Count
{I1, I2}	4
{I1, I3}	4
{I1, I4}	1
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2
{I3, I4}	0
{I3, I5}	1
{I4, I5}	0

C_2

Compare candidate support count with minimum support count

Itemset	Sup Count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

L_2

Step 2: Generating 2-itemset Frequent Pattern [Cont.]

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses L_1 *Join* L_1 to generate a candidate set of 2-itemsets, C_2.
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- Note: We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern

L2= [{I1,I2} {I1,I3}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}].

L3 = L2 JOIN L2 i.e.

A		B		C
{I1,I2}		{I1,I2}		
{I1,I3},	J	{I1,I3},		
{I1,I5},	O	{I1,I5},	=>	{I1,I2,I3},
{I2,I3},	I	{I2,I3},		{I1,I2,I5}.
{I2,I4},	N	{I2,I4},		
{I2,I5}].		{I2,I5}].		

Procedure Step 1 :

Find Items starting with I2 in B

It gives {I1, I2, I3}, {I1, I2, I4}, {I1, I2, I5}

Step 2 :

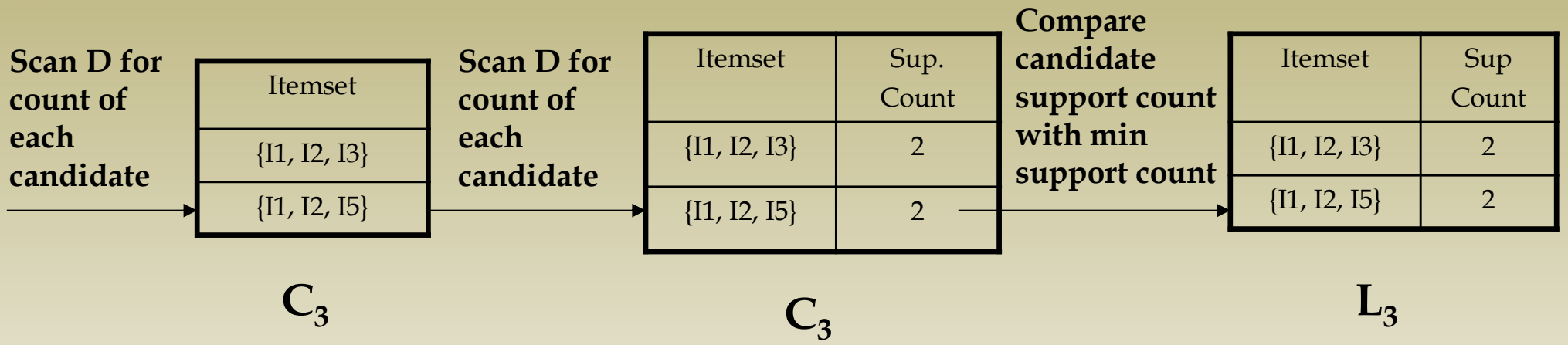
Find Items starting with I3 in B

It gives NIL, Similarly I4, I5

Step 3 :

Find out infrequent items sets using min support count and remove them.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3, we compute L_2 *Join* L_2.
- C_3 = L2 *Join* L2 = {{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3. Prune step helps to avoid heavy computation due to large C_k.

Step 3: Generating 3-itemset Frequent Pattern [Cont.]

- Based on the Apriori property that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take {I1, I2, I3}. The 2-item subsets of it are {I1, I2}, {I1, I3} & {I2, I3}. Since all 2-item subsets of {I1, I2, I3} are members of L_2, We will keep {I1, I2, I3} in C_3.
- Lets take another example of {I2, I3, I5} which shows how the pruning is performed. The 2-item subsets are {I2, I3}, {I2, I5} & {I3,I5}.
- BUT, {I3, I5} is not a member of L_2 and hence it is not frequent violating Apriori Property. Thus We will have to remove {I2, I3, I5} from C_3.
- Therefore, C_3 = {{I1, I2, I3}, {I1, I2, I5}} after checking for all members of result of Join operation for Pruning.
- Now, the transactions in D are scanned in order to determine L_3, consisting of those candidates 3-itemsets in C_3 having minimum support.

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses L_3 *Join* L_3 to generate a candidate set of 4-itemsets, C_4. Although the join results in {{I1, I2, I3, I5}}, this itemset is pruned since its subset {{I2, I3, I5}} is not frequent.
- Thus, $C_4 = \varphi$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?

  These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

- Procedure:
  - For each frequent itemset ***"l"***, generate all nonempty subsets of ***l***.
  - For every nonempty subset ***s*** of ***l***, output the rule **"s → (l-s)"** if **support_count(l) / support_count(s) >= min_conf** where min_conf is minimum confidence threshold.

- Back To Example:

We had L = {{I1}, {I2}, {I3}, {I4}, {I5}, {I1,I2}, {I1,I3}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}, {I1,I2,I3}, {I1,I2,I5}}.

  - Lets take ***l*** = {I1,I2,I5}.
  - Its all nonempty subsets are {I1,I2}, {I1,I5}, {I2,I5}, {I1}, {I2}, {I5}.

Step 5: Generating Association Rules from Frequent Itemsets [Cont.]

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.

  - R1: I1 ^ I2 → I5
    - Confidence = sc{I1,I2,I5}/sc{I1,I2} = 2/4 = 50%
    - R1 is Rejected.
  - R2: I1 ^ I5 → I2
    - Confidence = sc{I1,I2,I5}/sc{I1,I5} = 2/2 = 100%
    - R2 is Selected.
  - R3: I2 ^ I5 → I1
    - Confidence = sc{I1,I2,I5}/sc{I2,I5} = 2/2 = 100%
    - R3 is Selected.
  - R4: I1 → I2 ^ I5 confidence=2/6=33%
  - R5: I2 → I1 ^ I5 confidence=2/7=29%
  - R6: I5 → I1 ^ I2 confidence=2/2=100%

Step 5: Generating Association Rules from Frequent Itemsets [Cont.]

- R4: I1 → I2 ^ I5
  - Confidence = sc{I1,I2,I5}/sc{I1} = 2/6 = 33%
  - R4 is Rejected.
- R5: I2 → I1 ^ I5
  - Confidence = sc{I1,I2,I5}/{I2} = 2/7 = 29%
  - R5 is Rejected.
- R6: I5 → I1 ^ I2
  - Confidence = sc{I1,I2,I5}/ {I5} = 2/2 = 100%
  - R6 is Selected.

  In this way, We have found three strong association rules.

Methods to Improve Apriori's Efficiency

- **Hash-based Technique (Hashing itemset counts) :** A hash-based technique can be used to reduce the size of the candidate k-itemsets, Ck, for K>1. For ex, when scanning each transaction in the database to generate the frequent 1-itemsets, L1, from the candidate 1-itemsets in C1, we can generate all of the 2-itemsets for each transaction, hash (i.e. map) them into different buckets of a hash table structure, and increase the corresponding buckets count.

- A 2-itemset whose corresponding bucket count in the hash table is below the support threshold cannot be frequent and thus should be removed from the candidate set. Such a hash-based technique may substantially reduce the number of candidates k-itemsets examined.

Hash-based technique

Let us assume:

Item		Order
• I1	-	1
• I2	-	2
• I3	-	3
• I4	-	4
• I5	-	5

Bucket address	0	1	2	3	4	5	6
Bucket Count	2	2	4	2	2	4	4
Bucket Contents	{I1,I4} {I3,I5}	{I1,I5} {I1,I5}	{I2,I3} {I2,I3} {I2,I3} {I2,I3}	{I2,I4} {I2,I4}	{I2,I5}	{I1,I2} {I1,I2} {I1,I2}{ I1,I2}	{I1,I3} {I1,I3} {I1,I3} {I1,I3}

- Create hash table H2 using hash function
- **h(x,y) = ((order of x) x 10 + (order of y)) mod 7**
- From table of step 2 (of two item set) first take {I1, I2}
- Then apply formula like h(x,y) = ((1) x 10 + (2)) mod 7
- This gives 5. So put this in column 5 and increase bucket count by 1
- Then go the main table and find all {I1, I2) and do the same.

- **FORMULA = h(x,y) = ((order of x) x 10 + (order of)) mod 7**
- From table of step 2 (of two item set) first take {I1, I3}
- Then apply formula like h(x,y) = ((1) x 10 + (3)) mod 7
- This gives 6. So put this in column 6 and increase bucket count by 1
- Then go the main table and find all {I1, I3) and do the same.

- **FORMULA = h(x,y) = ((order of x) x 10 + (order of)) mod 7**
- From table of step 2 (of two item set) first take {I1, I4}
- Then apply formula like h(x,y) = ((1) x 10 + (4)) mod 7
- This gives 0. So put this in column 0 and increase bucket count by 1
- Then go the main table and find all {I1, I4) and do the same.

- Similarly do for the rest of the combinations

Bucket address	0	1	2	3	4	5	6
Bucket Count	2	2	4	2	2	4	4
Bucket Contents	{I1,I4} {I3,I5}	{I1,I5} {I1,I5}	{I2,I3} {I2,I3} {I2,I3} {I2,I3}	{I2,I4} {I2,I4}	{I2,I5}	{I1,I2} {I1,I2} {I1,I2}{I1,I2}	{I1,I3} {I1,I3} {I1,I3} {I1,I3}

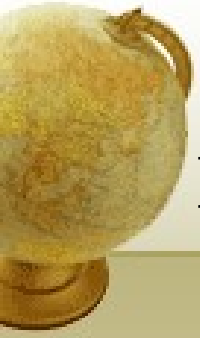
- This hash table was generated by scanning the transactions of previous figure while determining L1 from C1. If the minimum support count is say 3, then the itemsets in bucket 0,1,3, and 4 cannot be frequent and so they should not be included in C2.

Methods to Improve Apriori's Efficiency

- Hash-based itemset counting: A k-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
- Transaction reduction: A transaction that does not contain any frequent k-itemset is useless in subsequent scans
- Partitioning: Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
- Sampling: mining on a subset of given data, lower support threshold + a method to determine the completeness
- Dynamic itemset counting: add new candidate itemsets only when all of their subsets are estimated to be frequent

Is Apriori Fast Enough? — Performance Bottlenecks

- The core of the Apriori algorithm:
  - Use frequent (k – 1)-itemsets to generate candidate frequent k-itemsets
  - Use database scan and pattern matching to collect counts for the candidate itemsets
- The bottleneck of *Apriori*: candidate generation
  - Huge candidate sets:
    - 10^4 frequent 1-itemset will generate 10^7 candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \ldots, a_{100}\}$, one needs to generate $2^{100} \approx 10^{30}$ candidates.
  - Multiple scans of database:
    - Needs (n +1) scans, n is the length of the longest pattern



Mining Frequent Patterns Without Candidate Generation

- Compress a large database into a compact, Frequent-Pattern tree (FP-tree) structure
  - highly condensed, but complete for frequent pattern mining
  - avoid costly database scans
- Develop an efficient, FP-tree-based frequent pattern mining method
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only!

Example :

- We reexamine the mining of transaction database D.

- The first scan of the database is the same as Apriori, which derives the set of frequent items (1-itemsets) and their support counts (frequencies).

- Let the minimum support count be 2.

- The set of frequent items is stored in the order of descending support count.

- This resulting set or list is denoted as L.

- Thus, we have L = [I2:7, I1:6, I3:6, I4:2, I5:2],

- An FP-tree is then constructed as follows:

- First, create the root of the tree, labeled with “null”.
- Scan database D a second time. The items in each transaction are processed in L order (i.e. sorted according to descending support count) and a branch is created for each transaction.
- For ex, the scan of the first transaction, “T100 : I1,I2,I5 which contains three items (I2,I1,I5) in L order leads to the construction of the first branch of the tree with three nodes : (I2:1),(I1:1),(I5:1), where I2 is linked as a child of the root, I1 is linked to I2, and I5 is linked to I2
- In second transaction T200, contains the items I2 and I4 in L order, which would result in a branch were I2 is linked to the root and I4 is linked to I2.
- However this branch would share a common prefix, (I2), with the existing path for T100. Therefore we instead increment the count of the I2 node by 1, and create a new node (I4:1), which is linked as a child of (I2:2).
- In general, when considering the branch to be added for a transaction, the count of each node among a common prefix is incremented by 1, and nodes for the items following the prefix and created and linked accordingly

Steps:

1 Scan DB once, find frequent 1-itemset (single item pattern)

2 Order frequent items in frequency descending order

3 Scan DB again, construct FP-tree

“T100” : I1,I2,I5 -> (I2,I1,I5) in L order

“T200” : I2,I4 -> I2,I4 in L order

“T300” : I2,I3 -> I2,I3 in L order

“T400” : I1,I2,I4 -> I2,I1,I4 in L order

“T500” : I1,I3 -> I1,I3 in L order

“T600” : I2,I3 -> I2,I3 in L order

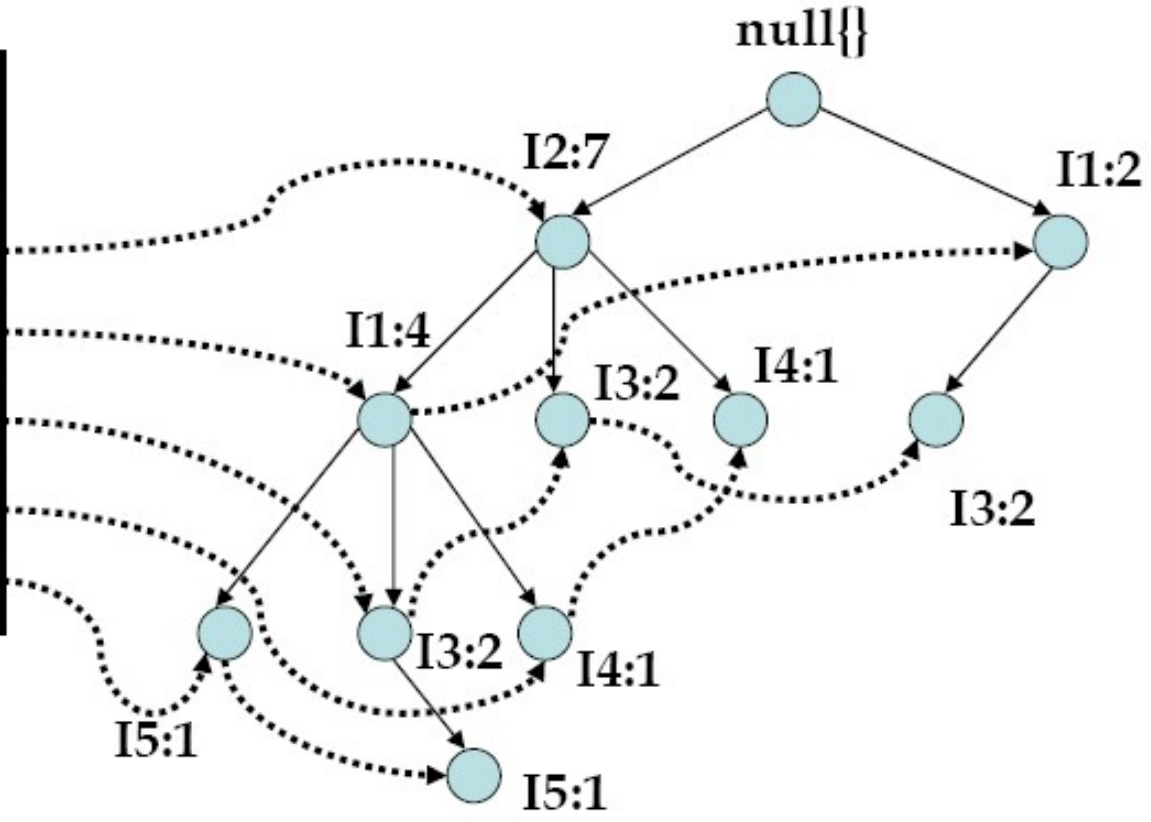
“T700” : I1,I3 -> I1,I3 in L order

“T800” : I1,I2,I3,I5 -> I2,I1,I3,I5 in L order

“T900” : I1,I2,I3 -> I2,I1,I3 in L order

FP-Growth Method: Construction of FP-Tree

Item Id	Sup Count	Node-link
I2	7	
I1	6	
I3	6	
I4	2	
I5	2	



An FP-Tree that registers compressed, frequent pattern information

Benefits of the FP-tree Structure

- Completeness:
  - never breaks a long pattern of any transaction
  - preserves complete information for frequent pattern mining
- Compactness
  - reduce irrelevant information—infrequent items are gone
  - frequency descending ordering: more frequent items are more likely to be shared
  - never be larger than the original database (if not count node-links and counts)
  - Example: For Connect-4 DB, compression ratio could be over 100

Mining Frequent Patterns Using FP-tree

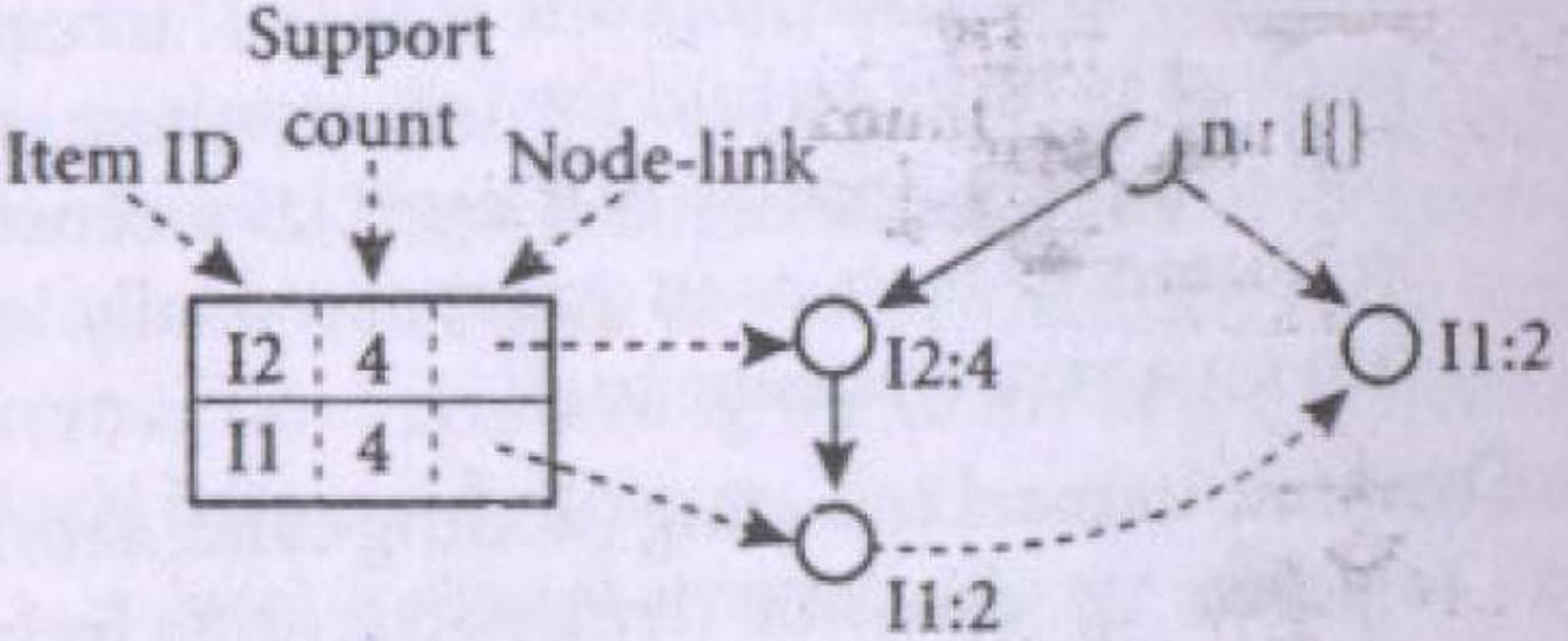
- General idea (divide-and-conquer)
  - Recursively grow frequent pattern path using the FP-tree
- Method
  - For each item, construct its conditional pattern-base, and then its conditional FP-tree
  - Repeat the process on each newly created conditional FP-tree
  - Until the resulting FP-tree is empty, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

Major Steps to Mine FP-tree

- Mining of he FP-Tree is summarized in the table below.
- Let us consider I5 which is the last item in L1, rather than the first.
- The reasoning behind this will become apparent as we explain the FP- Tree process.
- I5 occurs in two branches of the FP-Tree (Back 3 slides)
- The occurrences of I5 can easily be found by following its chain of node-links.
- The paths formed by these branches are <(I2 I1 I5 : 1)> and <(I2 I1 I3 I5 : 1)>
- Therefore considering I5 as a suffix, its corresponding two prefix paths are <(I2 I1 : 1)) and <(I2 I1 I3 :1)>, which from its conditional pattern base.
- Its conditional FP-Tree contains only a single path, <I2:2, I1:2) ; I3 is not included because its support count of 1 is less than the minimum support count.
- The single path generates all the combinations of frequent patterns : I2 I5:2,I1 I5:2, I2 I1 I5:2

Item	Conditional pattern base	Conditional FP-Tree	Frequent patterns generated
I5	{(I2 I1:1), (I2 I1 I3:1)}	<I2:2, I1:2>	I2 I5:2, I1 I5:2, I2 I1 I5:2
I4	{(I2 I1:1), (I2:1)}	<I2:2>	I2 I4:2
I3	{(I2 I1:2), (I2:2), (I1:2)}	<I2:4, I1:2>, <I1:2>	I2 I3:4, I1, I3:2, I2 I1 I3 :2
I1	{(I2 :4)}	<I2:4>	I2 I1:4

- For I4, its two prefix paths from the conditional pattern base, {(I2 I1 : 1), (I2 :1)} which generates a single node conditional FP-tree <I2:2> and derives one frequent pattern I2 I4 :2.
- Similarly I3's conditional pattern base is : {(I2 I1 :2), (I2:2), (I1:2)}.
- Its conditional FP-tree has two branches <I2:4, I1:2> AND <I1:2>, as shown in the figure below, which generates the set of patterns {I2 I3 :4, I1 I3:2, I2 I1 I3:2}
- Finally I1's conditional pattern base is {(I2:4)}, whose FP-tree contains only one node <I2:4>, which generates one frequent pattern I2 I1 : 4.



6.9 The conditional FP-tree associated with the conditional node

Major Steps to Mine FP-tree

- Construct conditional pattern base for each node in the FP-tree
- Construct conditional FP-tree from each conditional pattern-base
- Recursively mine conditional FP-trees and grow frequent patterns obtained so far
  - If the conditional FP-tree contains a single path, simply enumerate all the patterns

Why Is Frequent Pattern Growth Fast?

- Our performance study shows
  - FP-growth is an order of magnitude faster than Apriori, and is also faster than tree-projection
- Reasoning
  - No candidate generation, no candidate test
  - Use compact data structure
  - Eliminate repeated database scan
  - Basic operation is counting and FP-tree building

Iceberg Queries

- The Aprioiri algorithm can be used to improve the efficiency of answering iceberg queries.
- Icerberg query: Compute aggregates over one or a set of attributes only for those whose aggregate values is above certain threshold
- Example:

```
select P.custID, P.itemID, sum(P.qty)
from purchase P
group by P.custID, P.itemID
having sum(P.qty) >= 10
```

- Compute iceberg queries efficiently by Apriori:
  - First compute lower dimensions
  - Then compute higher dimensions only when all the lower ones are above the threshold

Example

- A database has five transactions. Let *min sup* = 60% and *min con f* = 80%.

  TID items bought
  T100 M, O, N, K, E, Y
  T200 D, O, N, K, E, Y
  T300 M, A, K, E
  T400 M, U, C, K, Y
  T500 C, O, O, K, I, E

  (a) Find all frequent itemsets using Apriori and FP-growth, respectively. compare the efficiency of the two mining processes.

  (b) List all of the *strong* association rules (with support *s* and confidence *c*) matching the following metarule, where *X* is a variable representing customers, and *item i* denotes variables representing items (e.g., *"A", "B"*, etc.):

  For every *x belongs to transaction*, *buys*(*X*, *item*1)^*buys*(*X*, *item*2))*buys*(*X*, *item*3) [*s*, *c*]

Mining Association Rules in Large Databases

- Association rule mining
- Mining single-dimensional Boolean association rules from transactional databases
- Mining multilevel association rules from transactional databases

Multiple-Level Association Rules

- For many applications, it is difficult to find strong associations among data items at low or primitive levels of abstractions due to sparsity of data in multidimensional space.
- Therefore, data mining systems should provide capabilities to mine association rules at multiple levels of abstraction and traverse easily among different abstraction space
- Lets examine the following example
- Suppose we are given the task relevant set of transactional data in Table (next slide) for sales at the computer department of an AllElectronics Branch.
- The concept hierarchy for the items is shown in next slide
- The concept hierarchy has four levels referred as levels 0,1,2,3.

TID	Items Purchased
T1	IBM desktop computer, Sony B/W printer
T2	Microsoft educational software, Microsoft Financial Software
T3	Logitech Mouse, Computer Accessory, Ergoway Wrist Pad
T4	IBM desktop computer, Microsoft Financial Software
T5	IBM desktop computer
.	

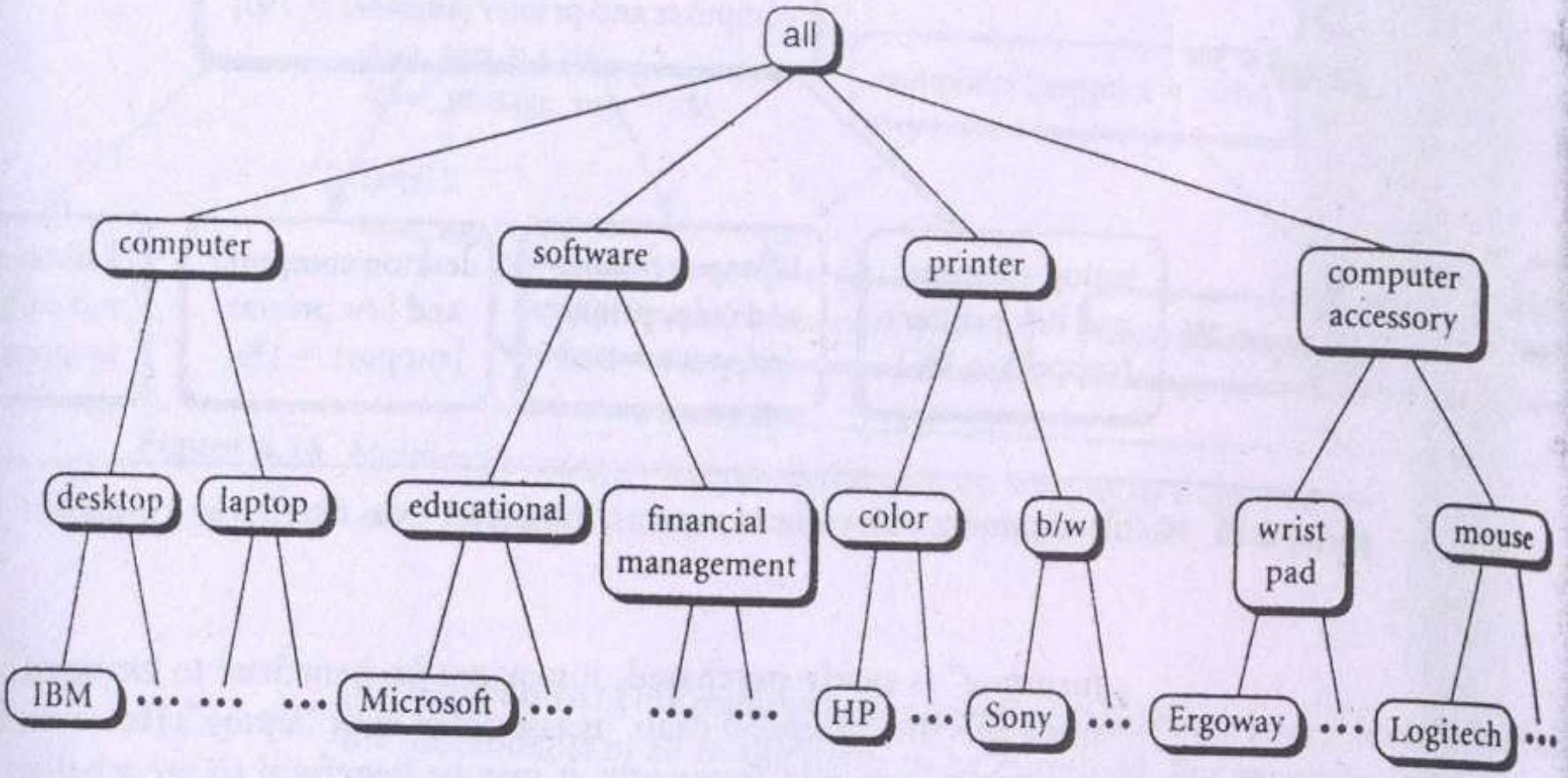


Figure 6.11 A concept hierarchy for *AllElectronics* computer items.

- By convention, levels within a concept hierarchy are numbered from top to bottom, starting with level 0 at the root node for all (the most general abstract level).

- Here level 1 includes computer, software, printer, and computer accessory
- Level 2 includes desktop computer, laptop computer, educational software, financial software etc.
- Level 3 includes IBM desktop computer, Microsoft software etc.
- Level 3 represents the most specific abstraction level of this hierarchy

- The items in Table are the lowest level of the concept hierarchy of Figure. It is difficult to find interesting purchase patterns at such raw or primitive level data.

- One would expect that it is easier to find strong association between “IBM desktop computer” and “B/W printer” rather than between “IBM desktop computer” and “Sony B/W printer

- Similarly, many people may purchase “computer” and “printer” together, rather than specifically purchasing “IBM desktop computer” and “Sony b/w printer” together.

- In other words, itemsets containing generalized items, such as “{IBM desktop computer, b/w printer}” replaced by {computer, printer} are easier to find interesting associations.

- Rules generated from association rule mining with concept hierarchies are called multi-level association rules.

Mining Multi-Level Associations

- A top down, progressive deepening approach is used :

  - First find high-level strong rules:
    milk ® bread [20%, 60%].

  - Then find their lower-level “weaker” rules:
    milk ® wheat bread [6%, 50%].

Multi-level Association: Uniform Support vs. Reduced Support

- Uniform Support: the same minimum support for all levels
  - + One minimum support threshold. No need to examine itemsets containing any item whose ancestors do not have minimum support.
  - – Lower level items do not occur as frequently. If support threshold
    - too high ⇒ miss low level associations
    - too low ⇒ generate too many high level associations

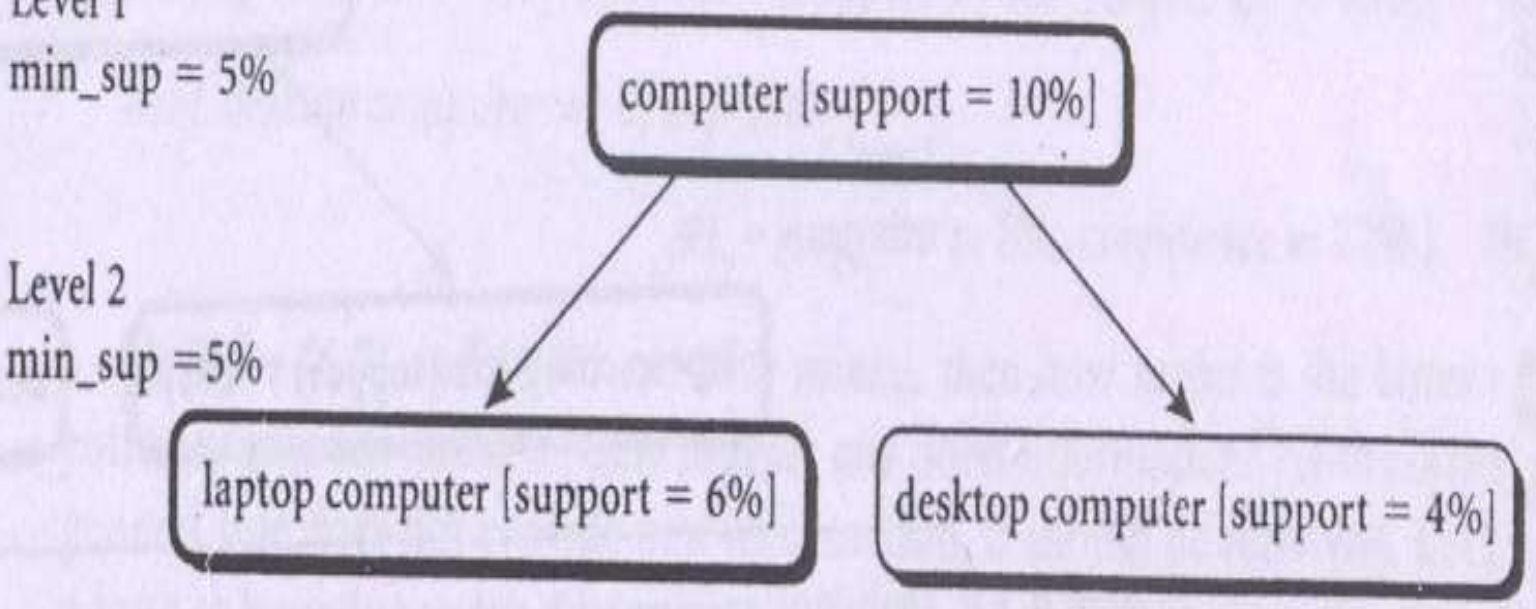


Figure 6.12 Multilevel mining with uniform support.

Multi-level Association: Uniform Support vs. Reduced Support

Reduced Support: reduced minimum support at lower levels

- There are 4 search strategies:
  - Level-by-level independent
  - Level-cross filtering by k-itemset
  - Level-cross filtering by single item
  - Controlled level-cross filtering by single item

Level-by-level independent : This is a full breadth search. Each node is examined, regardless of whether or not its parent node is found to be frequent

Level-cross filtering by single item : If a node is frequent, its children will be examined; otherwise, its descendent nodes are pruned from the search. For ex computer i.e. laptop and desktop are not examined since computer is not frequent

Level-cross filtering by k-itemset : The 2 itemset "{computer, printer}" is frequent, therefore the other nodes are examined

Controlled level-cross filtering by single item : A modified version of the level cross filtering by single item strategy. A threshold, called the level passage threshold can be set up for passing down relatively frequent item (called subfrequent items) to lower levels.

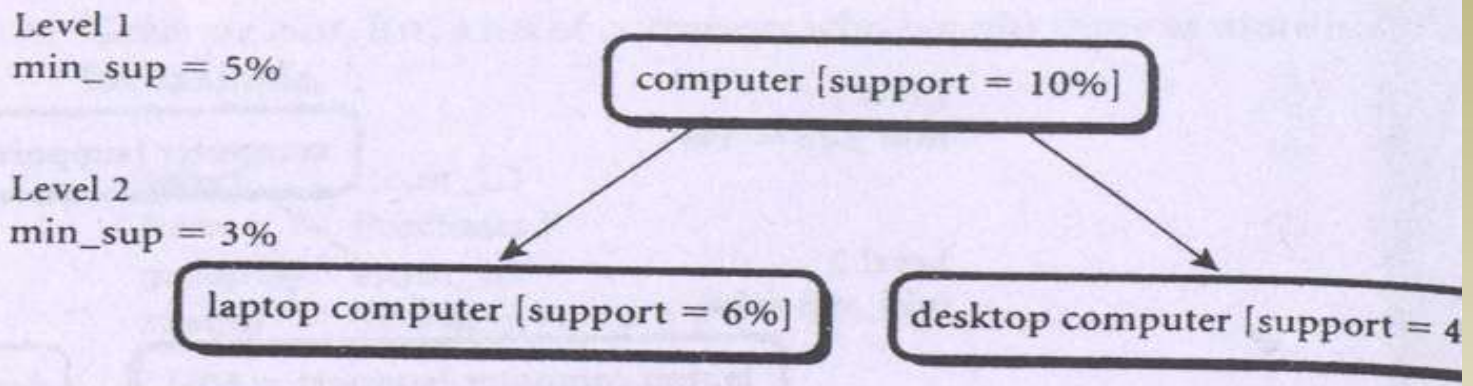


Figure 6.13 Multilevel mining with reduced support.

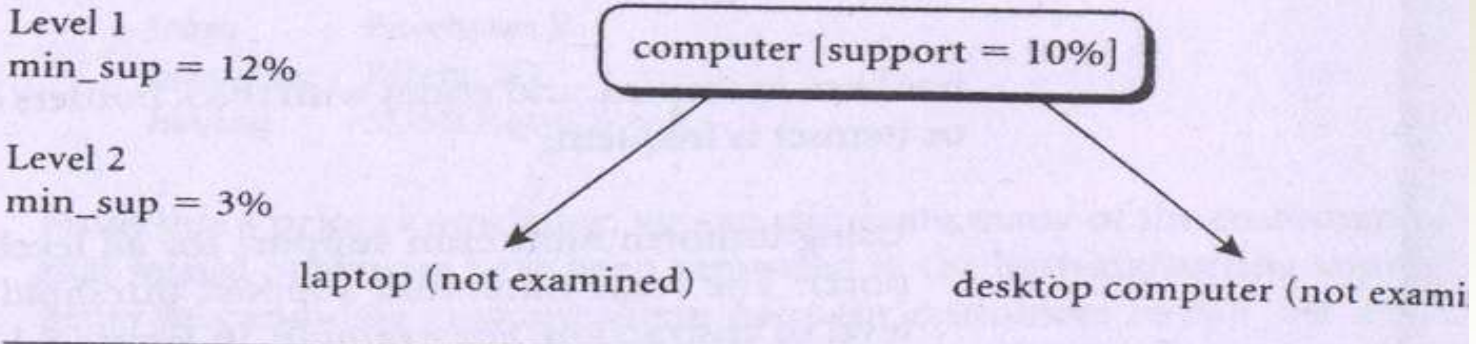


Figure 6.14 Multilevel mining with reduced support, using level-cross filtering by a single item.

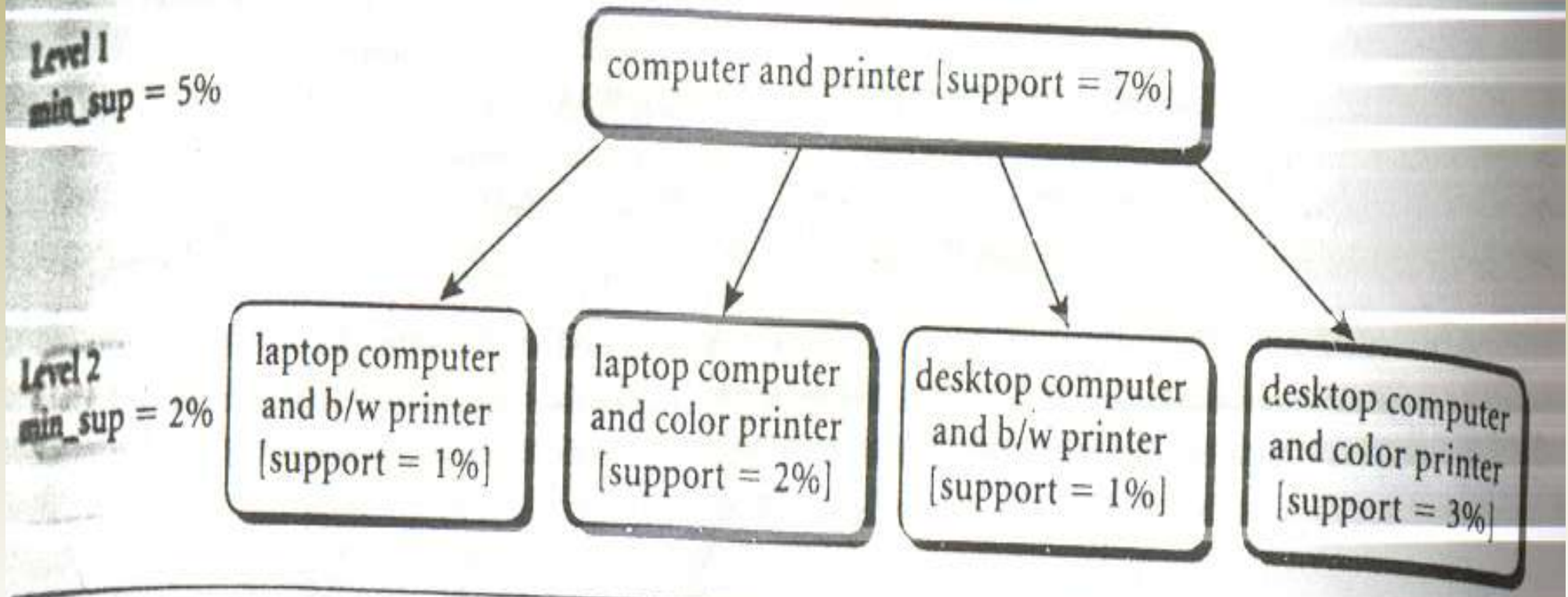
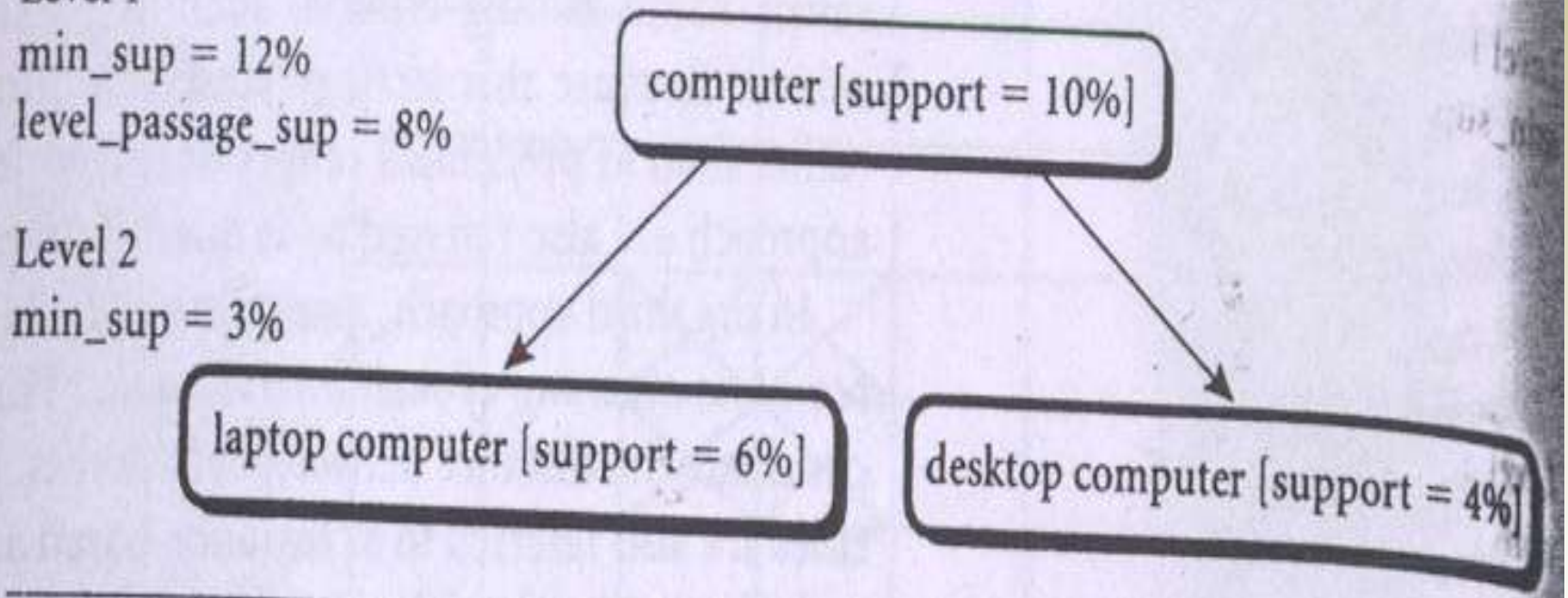


Figure 6.15 Multilevel mining with reduced support, using level-cross filtering by a k-itemset. Here, $k = 2$.

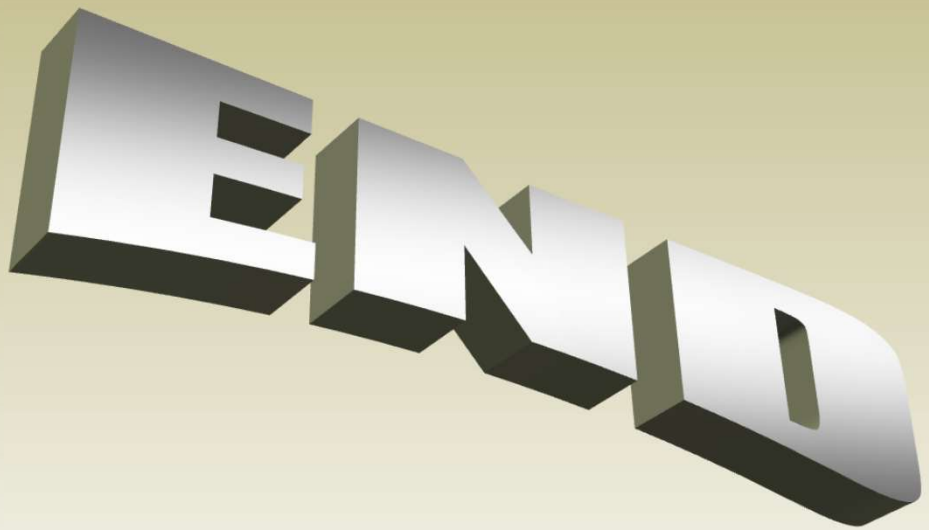


ure 6.16 Multilevel mining with controlled level-cross filtering by single item.

Multi-level Association: Redundancy Filtering

- Some rules may be redundant due to "ancestor" relationships between items.
- Example
  - Desktop computer ⇒ b/w printer [support = 8%, confidence = 70%]
  - IBM desktop computer ⇒ b/w printer [support = 2%, confidence = 72%]

- We say the first rule is an ancestor of the second rule.
- A rule is redundant if its support is close to the "expected" value, based on the rule's ancestor.


END